

Dossier d'Architecture

Plateforme Nationale d'État Civil

Auteur : Awade Bastien

1. Contexte et objectifs

La plateforme numérise le cycle de vie des actes d'état civil (naissance, mariage, décès) sur deux usages distincts :

- Enregistrement : les agents de mairie saisissent les actes au guichet.
- Consultation / délivrance : les usagers demandent leurs actes en ligne (extraits, copies intégrales).

1.1 Contraintes structurantes

	Contrainte	Nature	Impact architectural
C1	Connectivité instable en mairie (coupures de plusieurs heures)	Technique	Fonctionnement local autonome, non dépendant d'un lien permanent
C2	Pics de charge saisonniers (x10 en rentrée scolaire)	Technique	Isolement du flux de lecture pour protéger le flux d'écriture
C3	Non-perte et non-altération des actes	Réglementaire / métier	Écriture transactionnelle, journalisation immuable, intégrité vérifiable
C4	Conservation légale de long terme	Réglementaire	Archivage séparé, politique de rétention
C5	Souveraineté des données	Réglementaire	Hébergement 100% on-premise, aucun service cloud public

1.2 Lecture métier clé

L'enjeu réel n'est pas la volumétrie brute, c'est l'isolement : un pic de consultation ne doit jamais dégrader la capacité d'un agent à enregistrer une naissance. C'est cet isolement, pas la performance pure, qui justifie de séparer les deux chemins.

2. Vue d'ensemble de l'architecture cible

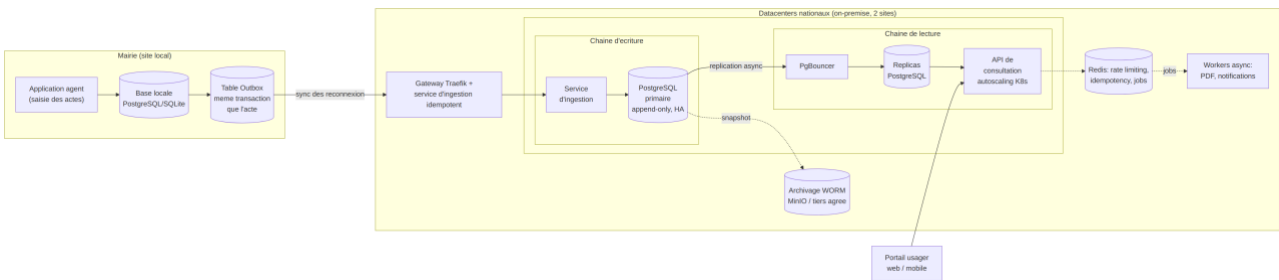


Figure 1 — Architecture cible : mairies en local-first, datacenters on-premise avec chaîne d'écriture et chaîne de lecture isolées.

Principe directeur : un chemin d'écriture protégé et isolé un chemin de lecture qui absorbe les pics sans jamais toucher la base d'écriture — obtenu par réplique PostgreSQL native, sans bus de message dédié.

3. Détail par composant

3.1 Niveau mairie — fonctionnement local-first (répond à C1)

Problème : un agent doit pouvoir enregistrer une naissance même sans réseau pendant plusieurs heures, sans jamais perdre la saisie ni la dupliquer au retour de la connexion.

Choix :

- Base locale embarquée (PostgreSQL) : l'acte est persisté immédiatement sur le poste, indépendamment du réseau.
- UUID généré côté client au moment de la saisie (pas côté serveur central), pour éliminer tout risque de collision d'identifiant lors de la synchronisation différée.
- Pattern Outbox transactionnel : l'acte et l'événement de synchronisation sont écrits dans la même transaction SQL locale, dans deux tables distinctes (actes et outbox). Un processus séparé lit outbox, pousse vers le central dès que le réseau revient, et supprime la ligne uniquement après accusé de réception. Cette atomicité garantit qu'il ne peut jamais exister un acte sans trace à synchroniser, ni l'inverse.

3.2 Synchronisation mairie ↔ central

Choix :

- Reverse proxy / API Gateway en frontal : Traefik — TLS, routage, rate limiting réseau. Rôle purement infrastructure.
- Reprise sur panne assurée par retry côté agent mairie (microservice dédié), et par les liveness/readiness probes de Kubernetes côté service d'ingestion pour le redémarrage automatique en cas de crash.
- Suppression de l'entrée Outbox locale uniquement après confirmation d'écriture durable côté central.
- Suivi du délai de synchronisation par mairie pour détecter les sites hors ligne anormalement longtemps.

3.3 Cœur central — isolement Write/Read par réplication

Chaîne d'écriture :

- Service d'ingestion : microservice applicatif conteneurisé, déployé sur un cluster Kubernetes on-premise. Son unique rôle : valider le payload reçu, effectuer l'INSERT idempotent dans PostgreSQL.
- PostgreSQL en append-only : aucune mise à jour destructive. Une correction crée un nouvel enregistrement lié à l'acte d'origine.
- Haute disponibilité par couple primaire/standby synchrone sur deux datacenters.
- Volume stable toute l'année : 2-3 réplicas pour la haute disponibilité, pas pour la charge. En cas de besoin d'autoscaling du HPA pourra être configuré pour le microservice afin d'absorber la charge.

Chaîne de lecture :

- Réplicas PostgreSQL en lecture seule, alimentés par réplication streaming native (asynchrone, quelques centaines de ms de retard acceptable pour de la consultation).
- PgBouncer (ou Pgpool-II) devant les réplicas : mutualise les connexions et répartit les requêtes de lecture entre plusieurs réplicas, évitant qu'un seul réplica sature.
- L'API de consultation ne lit jamais sur le primaire : elle interroge exclusivement les réplicas via PgBouncer.

3.4 Absorption des pics de charge — le facteur x10

Le pic de lecture est absorbé par la combinaison de quatre mécanismes :

- Autoscaling horizontal des pods API (HPA), déclenché sur des seuils cpu/mémoire définis. Les pods API sont stateless, donc le scaling ne nécessite aucune migration d'état.
- Pooling et répartition via PgBouncer devant les réplicas de lecture, pour mutualiser les connexions et lisser la charge entre réplicas.
- Réplicas supplémentaires activés en période de pic : nœuds physiques/VMs pré-configurés (Ansible/Terraform on-premise), joints au cluster de réplication juste avant la rentrée scolaire, puis retirés après — capacité saisonnière, pas permanente.
- Lissage côté traitements coûteux : génération de PDF signé et notifications partent en file Redis vers des workers asynchrones. L'API répond immédiatement ("demande prise en compte"), le travail lourd est exécuté en arrière-plan — évite que les pods API restent bloqués en attente d'E/S pendant le pic.

Traefik, en frontal, assure la répartition finale entre instances API.

3.5 Intégrité, non-altération et conservation légale

- Journalisation append-only dans la base d'écriture.
- Scellement cryptographique de chaque acte (hash chaîné, horodatage, signature électronique) pour prouver a posteriori l'absence d'altération.
- Archivage à froid on-premise : stockage objet auto-hébergé avec immuabilité ou recours à un tiers archiveur agréé totalement séparé de l'infrastructure opérationnelle.
- Politique de rétention documentée

4. Scénarios illustrés

4.1 Enregistrement en connectivité dégradée

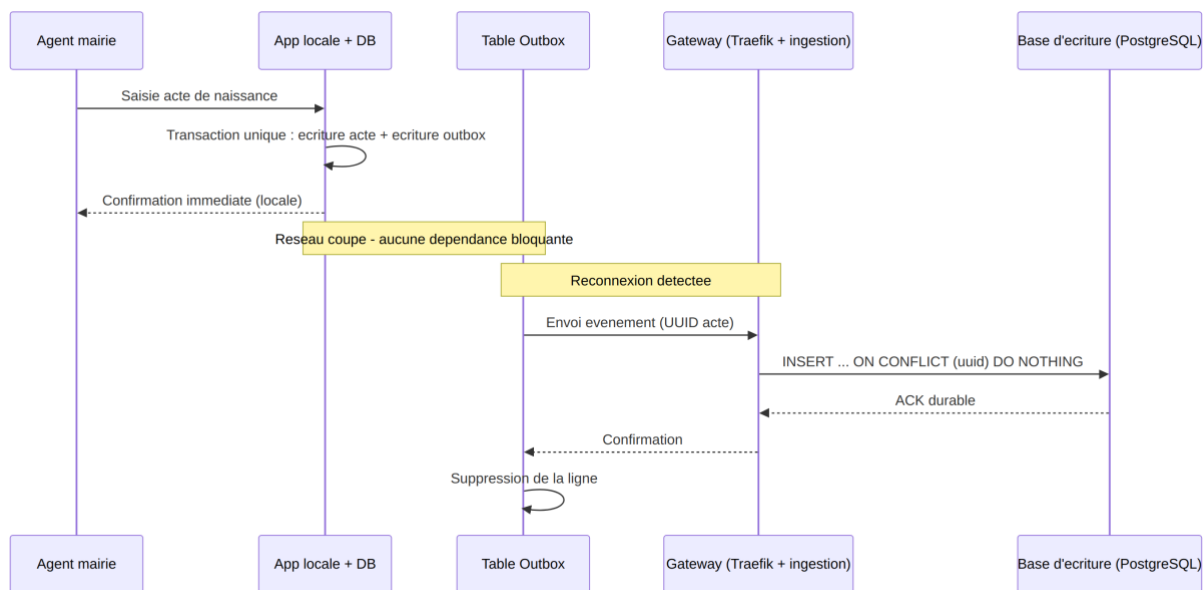


Figure 2 — Séquence : saisie locale, coupure réseau, synchronisation idempotente au retour de connexion.

4.2 Pic de charge en rentrée scolaire

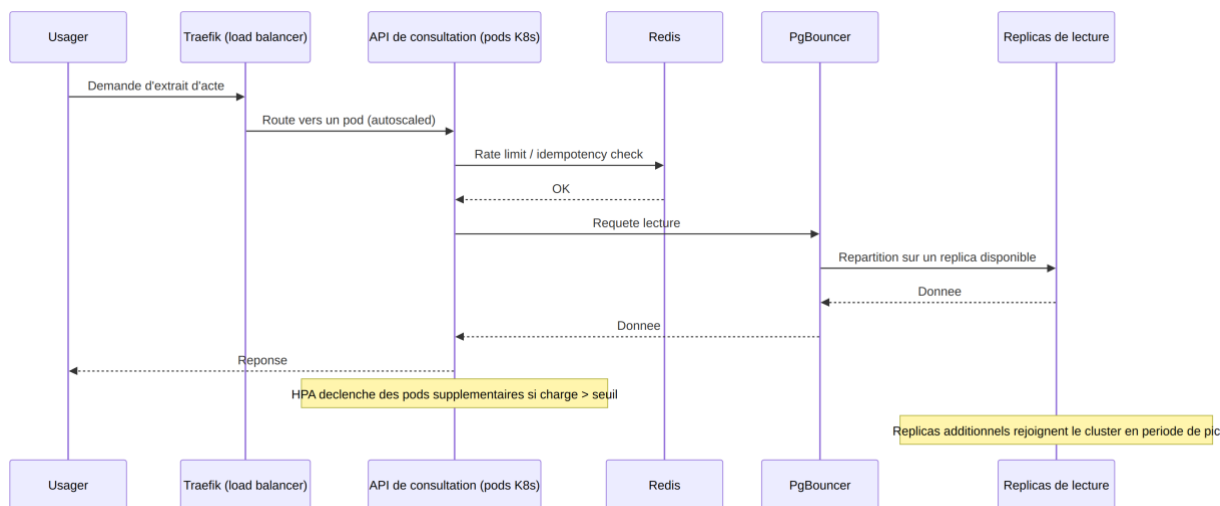


Figure 3 — Séquence : requête de consultation absorbée par le pooling, les réplicas de lecture et l'autoscaling.

5. Choix technologiques et justification

Composant	Outil proposé	Raison du choix
Base locale mairie	PostgreSQL	Cohérence transactionnelle forte requise même en local
Reverse proxy / gateway	Traefik	Léger, natif Kubernetes, TLS et rate limiting intégrés
Orchestration centrale	Kubernetes bare-metal (on-premise)	Autoscaling nécessaire, sans dépendance cloud
Service d'ingestion	Microservice	Simple, stateless
Base d'écriture	PostgreSQL (primaire + standby HA)	Maturité, garanties ACID, outillage d'audit
Isolement Write/Read	Réplication PostgreSQL native + PgBouncer	Suffisant pour le seul besoin d'isolement
Cache / opérations transverses	Redis (rate limiting, jobs)	Structures de données riches, adapté à la file de jobs légère
Archivage légal	Object Lock, ou tiers archiveur agréé	Immuabilité vérifiable, conformité, hébergement souverain
CI/CD	GitLab CI + ArgoCD (GitOps)	Déploiements traçables et auditables
Observabilité	Prometheus + Grafana (self-hosted)	Hébergeable en souverain, suivi de la dérive de sync par mairie

6. Exigences transverses

6.1 Sécurité

- Authentification forte des agents (MFA), traçabilité nominative de chaque écriture.
- Séparation des droits : un agent ne peut écrire que sur son périmètre communal.

6.2 Observabilité et exploitation

- Tableau de bord de suivi de synchronisation par mairie (dernier sync, événements en attente).
- Alerting sur dérive de synchronisation, saturation de la chaîne de lecture, échec de scellement d'un acte.

6.3 Reprise après incident

- Sauvegardes régulières testées de la base d'écriture, standby synchrone sur le second datacenter.
- Reconstruction d'un réplica de lecture par simple re-synchronisation depuis le primaire (source de vérité unique).

7 Plan de reprise (DR)

- PostgreSQL en cluster **primaire/standby géré par Patroni** (avec etcd pour l'élection de leader), répliqué entre les deux datacenters. Réplication synchrone si la latence inter-DC le permet (pour un RPO≈0) ; sinon semi-synchrone avec un RPO cible faible et documenté.
- **Failover automatique** : Patroni détecte la perte du primaire, promeut le standby, met à jour le endpoint. Les pods Kubernetes (service d'ingestion, API de lecture, PgBouncer) suivent le nouveau primaire sans intervention manuelle.
- Les mairies n'ont **rien à changer** côté client : la Gateway pointe vers une adresse stable (DNS interne / VIP), la bascule est transparente — l'Outbox continue simplement ses retries normaux pendant l'indisponibilité.

• 8. Sauvegarde

- Le risque réel n'est pas la base centrale, c'est le poste mairie isolé. Si le disque d'un poste tombe en panne avant que l'Outbox n'ait pu se synchroniser, les actes non encore transmis sont perdus. Mitigation : RAID local sur les postes des communes à connectivité la plus dégradée, et un export chiffré quotidien vers un support amovible en filet de sécurité supplémentaire pour ces sites à risque.
- L'archive WORM n'est pas une sauvegarde. Une sauvegarde sert à la reprise opérationnelle; l'archive légale sert à la conservation probante sur le temps long, avec un cycle de vie et des garanties d'immuabilité totalement séparés on ne restaure jamais depuis l'archive légale pour un incident technique courant.
- Tests de restauration : exercice de restauration complète documenté et tracé, cadence trimestrielle minimum une sauvegarde jamais testée n'est pas une sauvegarde.